

Internship Reflection

FedRAMP Assessment

Christian Bai | RISCPoint Advisory Group | Old Dominion University

The last three weeks at RISCPoint were spent on a FedRAMP penetration test for a federal cloud service provider. This was my first time working on a FedRAMP engagement from start to finish, and the scope was bigger than anything I had touched before. The client had over twenty externally facing web applications, a multi-tenant SaaS environment, and a full social engineering campaign built into the test plan. Testing ran from early to late March, and I was responsible for writing the final report.

The engagement kicked off with a phishing campaign targeting around thirty employees at the client organization. We registered a lookalike domain and built a fake Google Drive login page using GoPhish. The email pretended to share a document about company priorities for the year. Out of all thirty recipients, nobody submitted credentials, which was actually a solid result for the client. It was still a useful exercise for me because I got more practice setting up phishing infrastructure and tracking campaign metrics in GoPhish.

The web application testing is where things got interesting for me personally. The client's applications were all running on Microsoft IIS with ASP.NET backends, and I had not spent much time testing IIS environments before this engagement. A lot of the behavior was different from what I was used to seeing on Apache or Nginx. Things like ViewState parameters, ASPX page lifecycle quirks, and the way IIS handles error responses all required me to adjust my testing approach. One of the stack trace findings I wrote up actually came from submitting a modified ViewState value and watching the server spit back internal file paths from the build agent. That would not have happened the same way on a different stack.

The biggest learning experience for me was finding and writing up a DOM-based XSS vulnerability. I had done reflected and stored XSS testing before, but DOM-based was new territory. The vulnerable component was a help page viewer that read the URL query string with JavaScript and passed it into `document.write()` to build an iframe. The developers had tried to sanitize the input by stripping out literal colons and the hex-encoded version, but they missed the HTML entity ":" which the browser decoded back to a colon after the filter ran. That bypass let me inject a javascript URI that executed when the page loaded. What made it more impactful was that the same vulnerable

help viewer component was shared across five or six different applications in the client's suite, so one finding affected a large chunk of the attack surface.

Beyond the XSS work, I also identified improper authorization issues where low-privilege users could hit API endpoints meant for administrators, a stored XSS in a notification title field, insecure file uploads that accepted executables, session tokens being passed in URL parameters, and a CSRF vulnerability that could overwrite a user's security questions. The engagement ended with twelve total findings, three of which were rated high severity.

Writing the report for this one took more effort than past engagements because the FedRAMP format has specific requirements around how you document attack vectors and scope. Each mandatory attack vector from the FedRAMP Penetration Test Guidance had to be addressed individually, and the findings had to be tied back to the relevant vector. I spent a good amount of time making sure the evidence sections were thorough enough to stand up to a 3PAO review.

Overall this engagement pushed me in ways I did not expect. The IIS-specific testing and DOM-based XSS research were both areas where I had gaps in my knowledge going in, and I came out of the three weeks feeling a lot more confident in both. I also have a much better understanding of what goes into a FedRAMP-compliant pentest report, which I think will be useful.